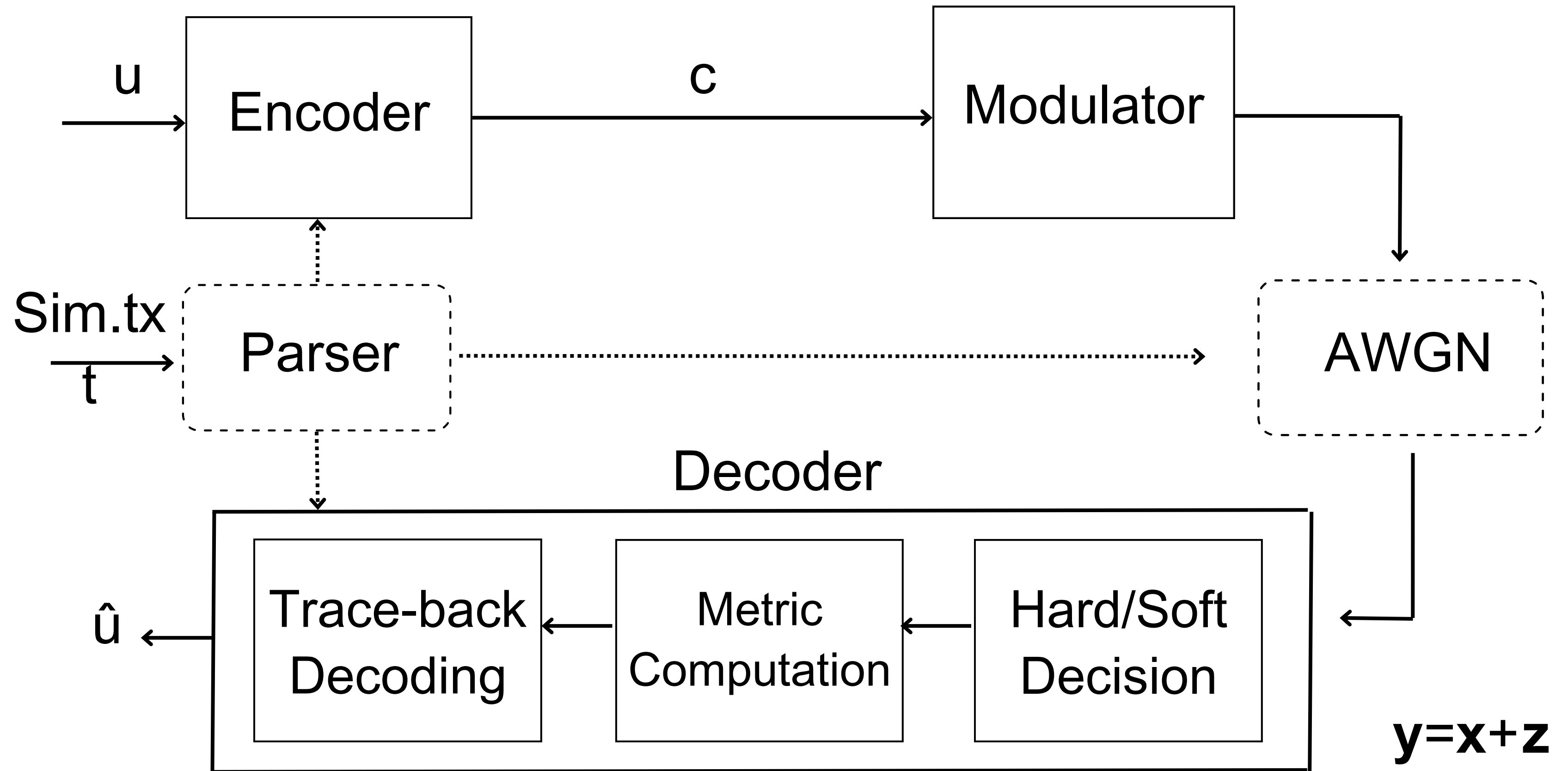


期中Project報告

F64106016 張博盛

Block Diagram of Convolutional Codes



Code流程

```
//decode
```

```
initialize_table();
```

```
if(decision == 0)
```

```
    hard_decision;
```

```
    viterbi_decoder_hard;
```

```
    writefile_hard;
```

```
else if(decision == 1)
```

```
    viterbi_decoder_soft;
```

```
    writefile_soft;
```

```
ber;
```

Code流程-Initialize Table

```
int g1 = 0b1101101;
```

```
int g2 = 0b1001111;
```

```
for state=0 to 63
```

```
  for input=0 to 1
```

```
    int shifted state = state << 1 + input;
```

```
    int next = shifted state & 111111;
```

```
    int out1 = __builtin_parity(shifted_state & g1);
```

```
    int out2 = __builtin_parity(shifted_state & g2);
```

```
    next_state_table[state][input] = next;
```

```
    output_table[state][input][0] = out1;
```

```
    output_table[state][input][1] = out2;
```

```
next_state_table[0][1] = 1
```

```
//state0 input=1 next state=1
```

```
output_table[0][1][0] = 1
```

```
//state 0 input=1 x1_output=1
```

```
output_table[0][1][0] = 1
```

```
//state 0 input=1 x2_output=1
```

Code流程-Initialize Table

State Table:

```
State: 0 (000000), Input: 0 -> Next State: 0 (000000), Output: 00
State: 0 (000000), Input: 1 -> Next State: 1 (000001), Output: 11
State: 1 (000001), Input: 0 -> Next State: 2 (000010), Output: 01
State: 1 (000001), Input: 1 -> Next State: 3 (000011), Output: 10
State: 2 (000010), Input: 0 -> Next State: 4 (000100), Output: 11
State: 2 (000010), Input: 1 -> Next State: 5 (000101), Output: 00
State: 3 (000011), Input: 0 -> Next State: 6 (000110), Output: 10
State: 3 (000011), Input: 1 -> Next State: 7 (000111), Output: 01
State: 4 (000100), Input: 0 -> Next State: 8 (001000), Output: 11
State: 4 (000100), Input: 1 -> Next State: 9 (001001), Output: 00
State: 5 (000101), Input: 0 -> Next State: 10 (001010), Output: 10
State: 5 (000101), Input: 1 -> Next State: 11 (001011), Output: 01
State: 6 (000110), Input: 0 -> Next State: 12 (001100), Output: 00
State: 6 (000110), Input: 1 -> Next State: 13 (001101), Output: 11
State: 7 (000111), Input: 0 -> Next State: 14 (001110), Output: 01
State: 7 (000111), Input: 1 -> Next State: 15 (001111), Output: 10
State: 8 (001000), Input: 0 -> Next State: 16 (010000), Output: 00
State: 8 (001000), Input: 1 -> Next State: 17 (010001), Output: 11
State: 9 (001001), Input: 0 -> Next State: 18 (010010), Output: 01
State: 9 (001001), Input: 1 -> Next State: 19 (010011), Output: 10
State: 10 (001010), Input: 0 -> Next State: 20 (010100), Output: 11
State: 10 (001010), Input: 1 -> Next State: 21 (010101), Output: 00
State: 11 (001011), Input: 0 -> Next State: 22 (010110), Output: 10
State: 11 (001011), Input: 1 -> Next State: 23 (010111), Output: 01
State: 12 (001100), Input: 0 -> Next State: 24 (011000), Output: 11
State: 12 (001100), Input: 1 -> Next State: 25 (011001), Output: 00
State: 13 (001101), Input: 0 -> Next State: 26 (011010), Output: 10
State: 13 (001101), Input: 1 -> Next State: 27 (011011), Output: 01
State: 14 (001110), Input: 0 -> Next State: 28 (011100), Output: 00
State: 14 (001110), Input: 1 -> Next State: 29 (011101), Output: 11
State: 15 (001111), Input: 0 -> Next State: 30 (011110), Output: 01
State: 15 (001111), Input: 1 -> Next State: 31 (011111), Output: 10
State: 16 (010000), Input: 0 -> Next State: 32 (100000), Output: 10
```

Next State:

```
0 1
2 3
4 5
6 7
8 9
10 11
12 13
14 15
16 17
18 19
20 21
22 23
24 25
26 27
28 29
30 31
32 33
34 35
36 37
38 39
40 41
42 43
44 45
46 47
48 49
50 51
52 53
54 55
56 57
```

Code流程-Viterbi_decode

以下為Viterbi decode的Pseudo Code，若為soft decision則將int改為double

```
int path[num_states][L+1]; //記錄走過的state
int metric[64][2]; //紀錄每個路徑的metric
for i=0 to 63 //初始化矩陣
    for j=0 to 1
        metric[i][j] = (i == 0 && j == 0) ? 0 : INT_MAX;
    for k = 0 to 32
        path[i][j] = (i == 0 && j == 0) ? 0 : -2;

for step = 0 to N+m-L //做N+m-L+1次
    ...
```

Code流程-Viterbi_decode

```
for step = 0 to N+m-L //做N+m-L+1次
    int temp_metric; //存取運算完的metric

    if(step == 0) //做第一個32bits的window，之後每次都只需做新一步的Column
        for bit=0 to 31
            for state=0 to 63
                if(path[state][bit] == -2) continue; //-2為不能走的路
                for input = 0 to 1
                    MetricComputation
                //將y和output_table中對應到的值做xor，如果是soft則用Euclidean distance
```

Code流程-Viterbi_decode

```
if(state < 32)
```

```
    metric[next][1] = metric[state][0] +temp_metric;
```

```
    path[state][bit+1] = state;
```

```
else if(state >= 32)
```

```
    if(metric[next0][0] + temp_metric < metric[next0][1])
```

```
        metric[next0][1] = metric[state][0] +temp_metric;
```

```
        path[state][bit+1] = state;
```

```
ShiftLeftMetic;
```

Code流程-Viterbi_decode

```
else if(step > 0 && step < N-L+m+1)
```

```
...
```

```
//與第一個window大同小異，差別差在只做最後一個Column
```

```
//find best state
```

```
int best_state = 0;
```

```
int min_mertic = INT_MAX;
```

```
for i=0 to 63
```

```
if(min_mertic > metric[i][0]){
```

```
    min_mertic = metric[i][0];
```

```
    best_state = i;
```

Code流程-Viterbi_decode

//尋找previous state

```
int prev_state[L];
```

```
prev_state[L-1] = best_state;
```

```
for bit = 32 to 1
```

```
    best_state = path[best_state][bit];
```

```
    prev_state[bit-2] = best_state;
```

```
if(prev_state[0] %2 == 0) decoded_bits[step] = 0;
```

```
else if (prev_state[0] %2 == 1) decoded_bits[step] = 1;
```

```
if(step == N+m-L)
```

```
for i to 31
```

```
    if(prev_state[0] %2 == 0) decoded_bits[step + i] = 0;
```

```
    else if (prev_state[0] %2 == 1) decoded_bits[step + i] = 1;
```

Parameter set

Output Decisioon	Best state
Truncation window length	L = 32、 64
No. information bits	10^6
Random seed	-2023

Simulation Data-Hard

SNR(dB)	BER(L=32)	BER(L=64)
0.0	3.78e-01	3.75e-01
0.5	3.26e-01	3.22e-01
1.0	2.63e-01	2.56e-01
1.5	1.92e-01	1.84e-01
2.0	1.27e-01	1.18e-01
2.5	7.28e-02	6.55e-02
3.0	3.70e-02	3.21e-02

Simulation Data-Hard

SNR(dB)	BER(L=32)	BER(L=64)
3.5	1.55e-02	1.30e-02
4.0	5.89e-03	4.94e-03
4.5	2.13e-03	1.73e-03
5.0	6.38e-04	5.57e-04
5.5	1.95e-04	1.60e-04
6.0	3.50e-05	3.20e-05

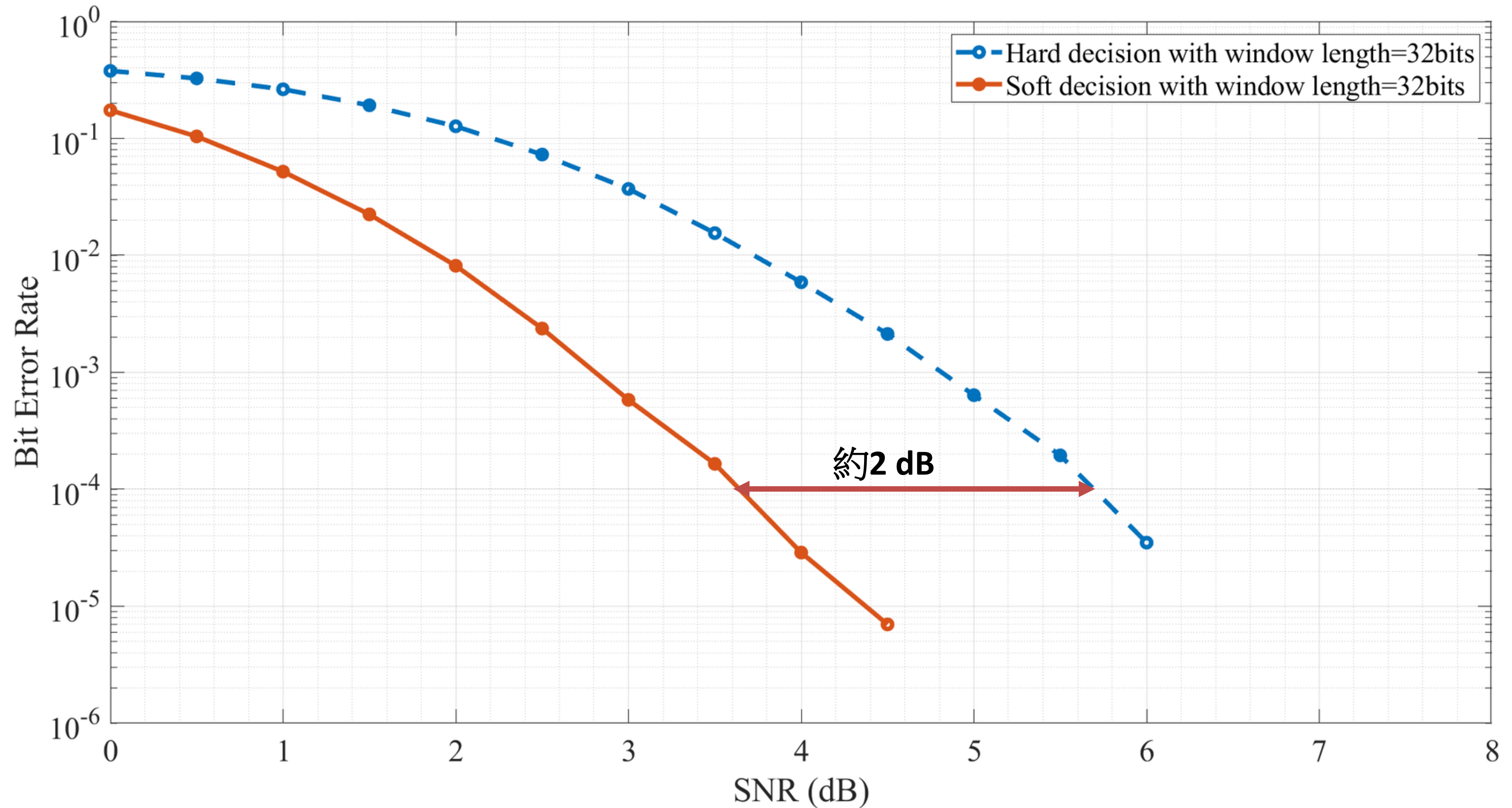
Simulation Data-Soft

SNR(dB)	BER(L=32)	BER(L=64)
0.0	1.74e-01	1.59e-01
0.5	1.04e-01	8.95e-02
1.0	5.20e-02	4.12e-02
1.5	2.24e-02	1.66e-02
2.0	8.15e-03	5.56e-03
2.5	2.37e-03	1.61e-03
3.0	5.82e-04	3.83e-04

Simulation Data-Soft

SNR(dB)	BER(L=32)	BER(L=64)
3.5	1.65e-04	1.07e-04
4.0	2.88e-05	2.00e-05
4.5	7.00e-06	7.00e-06
5.0	5.00e-06	5.00e-06

Simulation data with L=32



Simulation data with L=64

